

11 – Express

Application framework for Node.js

Jérôme Landré – jerome.landre@ju.se

OUTLINE

- 1) Express.js
- 2) Installation
- 3) Hello World
- 4) Middleware Architecture
- 5) Comparing Node and Express
- 6) Routes

1) EXPRESS.JS

- Express is a back-end application **framework** for Node.js
- It **simplifies** code development with Node.js

2) INSTALLATION

- In a project folder (where there is a "**package.json**" file)
- Run the command "**npm install express**"
- It will create a "**node_modules**" folder with many directories/files in it
- One of them being **express** and the others dependencies necessary to run express

```
C:\Users\lanjer\Desktop\my-node-app>npm install express

added 58 packages, and audited 59 packages in 1s

8 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

C:\Users\lanjer\Desktop\my-node-app>
```

```
C:\Users\lanjer\Desktop\my-node-app>dir node_modules
Volume in drive C is Windows
Volume Serial Number is 2671-B1FB

Directory of C:\Users\lanjer\Desktop\my-node-app\node_modules

2023-09-12  09:37    <DIR>          .
2023-09-12  09:37    <DIR>          ..
2023-09-12  09:37    <DIR>          .bin
2023-09-12  09:37         22 137 .package-lock.json
2023-09-12  09:37    <DIR>          accepts
2023-09-12  09:37    <DIR>          array-flatten
2023-09-12  09:37    <DIR>          body-parser
2023-09-12  09:37    <DIR>          bytes
```

```
2023-09-12  09:37    <DIR>          express

2023-09-12  09:37    <DIR>          statuses
2023-09-12  09:37    <DIR>          toidentifier
2023-09-12  09:37    <DIR>          type-is
2023-09-12  09:37    <DIR>          unpipe
2023-09-12  09:37    <DIR>          utils-merge
2023-09-12  09:37    <DIR>          vary
                1 File(s)          22 137 bytes
                60 Dir(s)  19 878 912 000 bytes free

C:\Users\lanjer\Desktop\my-node-app>
```

3) HELLO WORLD

- Let's do a Hello World app in Express

```
JS server-hello.js > ...
1   const express = require('express')
2   const app = express()
3   const port = 3000
4
5   app.get('/', (req, res) => {
6     res.send('Hello World!')
7   })
8
9   app.listen(port, () => {
10    console.log(`Example app listening on port ${port}`)
11  })
12
```

3) HELLO WORLD

- Run and check the result

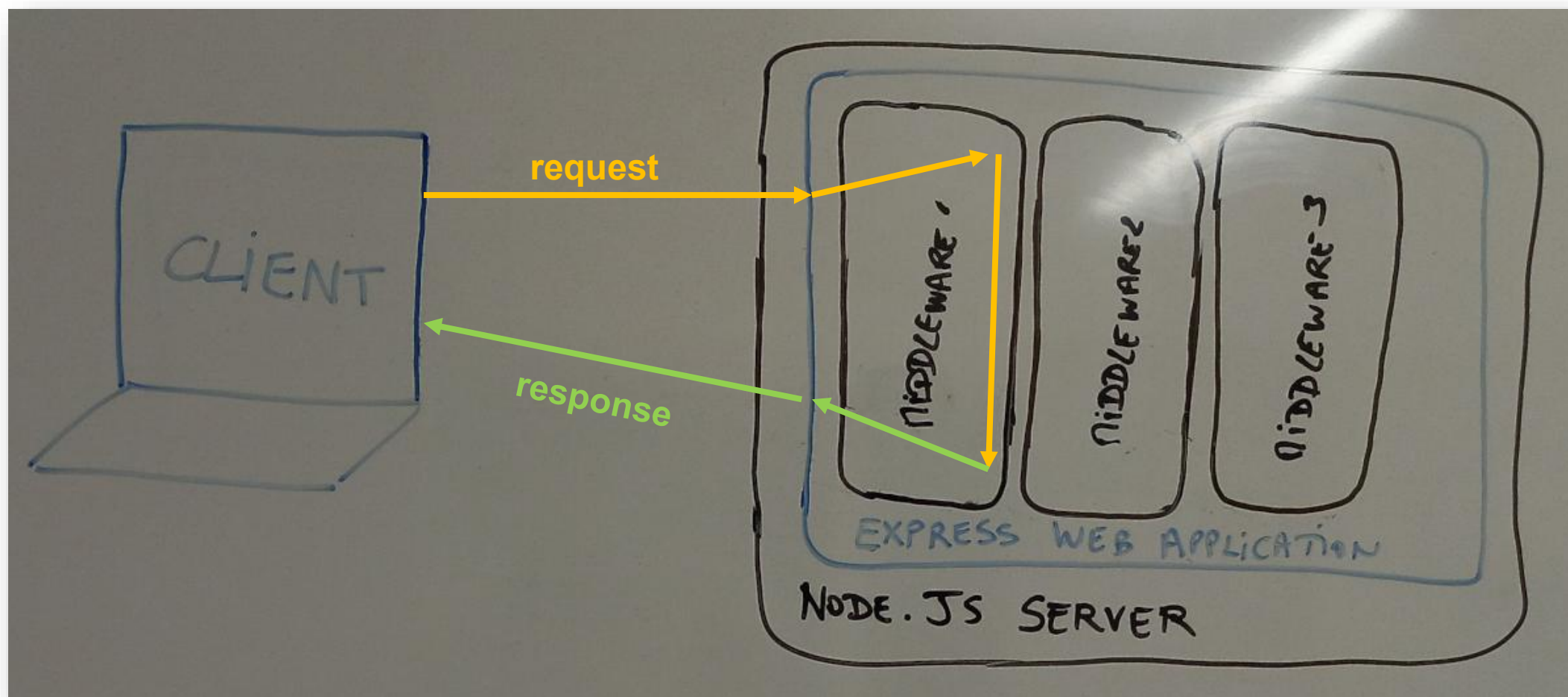
```
C:\Users\lanjer\Desktop\my-node-app>node server-hello.js  
Example app listening on port 3000
```



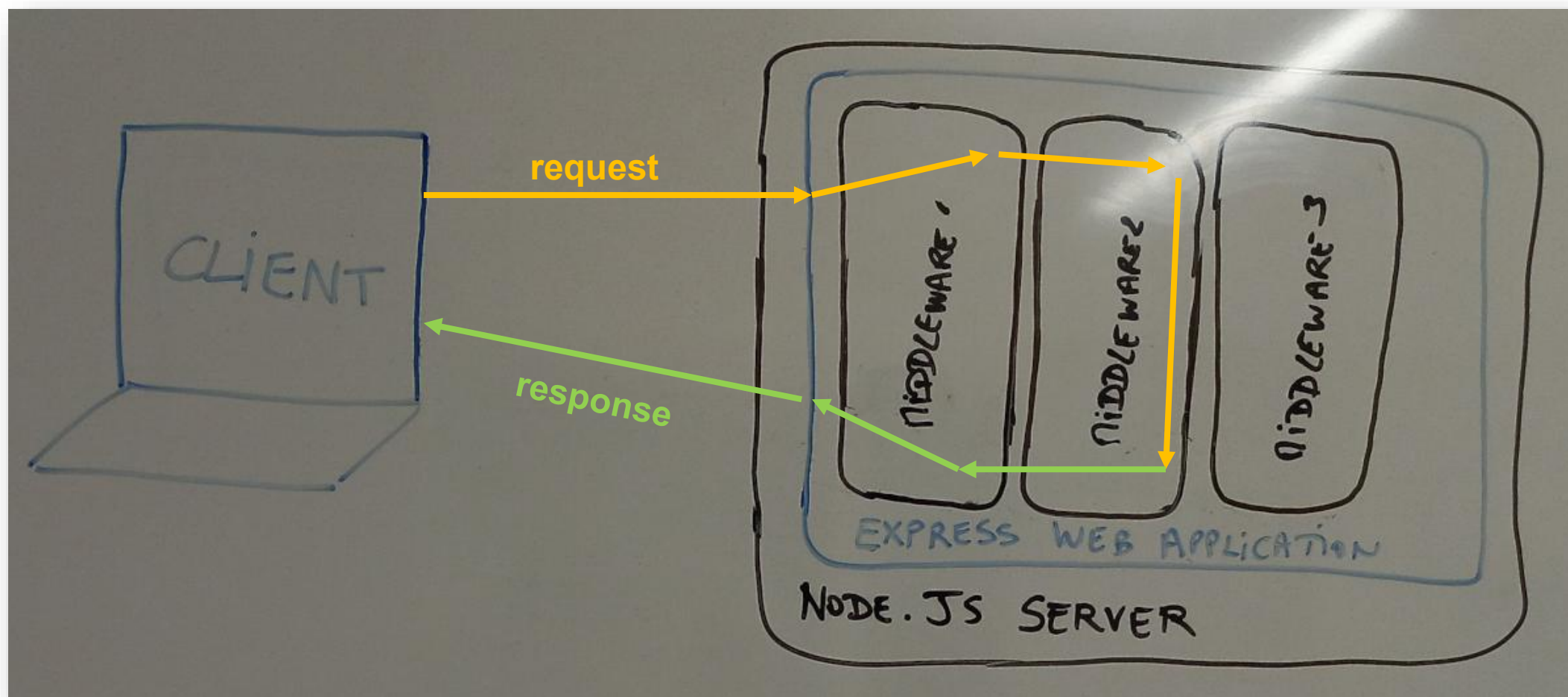
4) MIDDLEWARE ARCHITECTURE

- Express uses a **middleware architecture** in which the **request** passes through **possibly many middlewares (functions)** before building and sending the **response** to the client

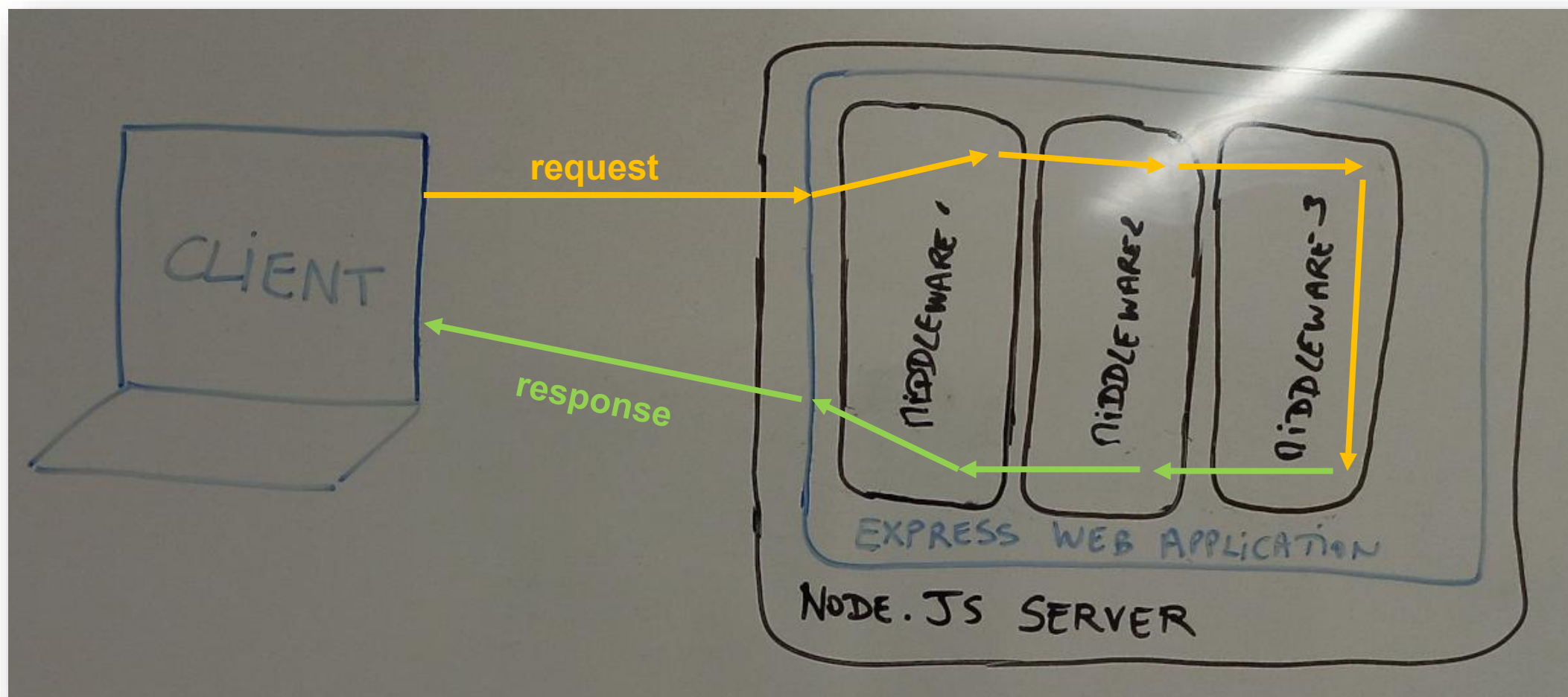
4) MIDDLEWARE ARCHITECTURE



4) MIDDLEWARE ARCHITECTURE



4) MIDDLEWARE ARCHITECTURE



4) MIDDLEWARE ARCHITECTURE

```
1  const express = require('express')
2  const app = express()
3
4  const port = 3000;
5
6  app.use(function(request, response, next) {
7    if(request.method == "GET" && request.uri == "/") {
8      response.send("Welcome on the / route!")
9    } else {
10     next()
11   }
12 });
13
14 app.listen(port, () => {
15   console.log(`Server running and listening on port ${port}...`);
16 });
17
```

Documentation:

<https://expressjs.com/en/4x/api.html#app>

Source: code from Peter LARSSON-GREEN

```
1  const express = require('express')
2  const app = express()
3  const port = 3000;
4
5  app.use(function(request, response, next) {
6    if(request.method == "GET" && request.uri == "/") {
7      response.send("Welcome on the / route!")
8    } else {
9      next()
10   }
11 });
12 app.use(function(request, response, next) {
13   if(request.method == "GET" && request.uri == "/hello") {
14     response.send("Hello!")
15   } else {
16     next()
17   }
18 });
19
20 app.listen(port, () => {
21   console.log(`Server running and listening on port ${port}...`);
22 });
23
```

```
1  const express = require('express')
2  const app = express()
3  const port = 3000;
4
5  app.use(function(request, response, next) {
6    if(request.method == "GET" && request.uri == "/") {
7      response.send("Welcome on the / route!")
8    } else {
9      next()
10   }
11 });
12 app.use(function(request, response, next) {
13   if(request.method == "GET" && request.uri == "/hello") {
14     response.send("Hello!")
15   } else {
16     next()
17   }
18 });
19
20 app.listen(port, () => {
21   console.log(`Server running and listening on port ${port}...`);
22 });
23
```

```
1  const express = require('express')
2  const app = express()
3  const port = 3000;
4
5  app.get("/", function (request, response) {
6    response.send("Welcome on the / route!")
7  });
8  app.get("/hello", function (request, response) {
9    response.send("Hello!")
10 });
11
12 app.listen(port, () => {
13   console.log(`Server running and listening on port ${port}...`);
14 });
15
```

4) MIDDLEWARE ARCHITECTURE

```
1  const express = require('express')
2  const app = express()
3  const port = 3000;
4
5  app.use(function(request, response, next){
6    request.myOwnVariable = "Jérôme, 1972."
7    next()
8  });
9
10 app.get("/hello", function (request, response) {
11   response.send("Hello "+request.myOwnVariable)
12 });
13
14 app.listen(port, () => {
15   console.log(`Server running and listening on port ${port}...`);
16 });
17
```



4) MIDDLEWARE ARCHITECTURE

- **Middleware** documentation:

<https://expressjs.com/en/resources/middleware.html>

- **Request** documentation:

<https://expressjs.com/en/api.html#req>

- **Response** documentation:

<https://expressjs.com/en/api.html#res>

5) COMPARING NODE AND EXPRESS

```
1  const http = require('http');
2
3  const port = 3000;
4
5  const server = http.createServer((req, res) => {
6    res.statusCode = 200;
7    res.setHeader('Content-Type', 'text/plain');
8    res.end('Hello World!');
9  });
10
11 server.listen(port, () => {
12   console.log(`Server running and listening on port ${port}...`);
13 });
14
```

```
1  const express = require('express')
2  const app = express()
3  const port = 3000
4
5  app.get('/', (req, res) => {
6    res.status(200).type("text/plain").send('Hello World!')
7  })
8
9  app.listen(port, () => {
10   console.log(`Example app listening on port ${port}`)
11 })
12
```


6) ROUTES

- In the code, `app.get("/", ... )`
- **app** is your **web server (application)**
- **get** is the method the server will receive onto
- `"/"` is the **route**

```
1  const express = require('express')
2  const app = express()
3  const port = 3000
4
5  app.get('/', (req, res) => {
6    | res.status(200).type("text/plain").send('Hello World!')
7    | })
8
9  app.listen(port, () => {
10   | console.log(`Example app listening on port ${port}`)
11   | })
12
```

6) ROUTES

- Your routes will define your **REST API** (it is the **CRUD** in your database)
 - **post**: create new information (Create)
 - **get**: get information (Read)
 - **put**: update information (Update)
 - **delete**: delete information (Delete)

```
1  const express = require('express')
2  const app = express()
3  const port = 3000
4
5  app.get('/', (req, res) => {
6    res.status(200).type("text/plain").send('Welcome to my book application!')
7  })
8
9  app.get("/books", (req, res) => {
10   console.log("List of all the books requested...")
11 })
12
13 app.post("/books/new", (req, res) => {
14   // with DATA
15   console.log("New book creation requested...")
16 })
17
18 app.delete("/books/234", (req, res) => {
19   console.log("Book 234 deletion requested...")
20 })
21
22 app.put("/books/234", (req, res) => {
23   // with DATA
24   console.log("Book 234 modification requested...")
25 })
26
27 app.listen(port, () => {
28   console.log(`Example app listening on port ${port}`)
29 })
30
```

6) ROUTES

- Your routes will define your **REST API** (it is the **CRUD** in your database)
 - **post**: create new information (Create)
 - **get**: get information (Read)
 - **put**: update information (Update)
 - **delete**: delete information (Delete)



ANY QUESTIONS?